



MixStyle Neural Networks for Domain Generalization and Adaptation

Kaiyang Zhou¹ · Yongxin Yang² · Yu Qiao³ · Tao Xiang⁴

Received: 8 November 2022 / Accepted: 15 September 2023 / Published online: 17 October 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Neural networks do not generalize well to unseen data with domain shifts—a longstanding problem in machine learning and AI. To overcome the problem, we propose MixStyle, a simple plug-and-play, parameter-free module that can improve domain generalization performance without the need to collect more data or increase model capacity. The design of MixStyle is simple: it mixes the feature statistics of two random instances in a single forward pass during training. The idea is grounded by the finding from recent style transfer research that feature statistics capture image style information, which essentially defines visual domains. Therefore, mixing feature statistics can be seen as an efficient way to synthesize new domains in the feature space, thus achieving data augmentation. MixStyle is easy to implement with a few lines of code, does not require modification to training objectives, and can fit a variety of learning paradigms including supervised domain generalization, semi-supervised domain generalization, and unsupervised domain adaptation. Our experiments show that MixStyle can significantly boost out-of-distribution generalization performance across a wide range of tasks including image recognition, instance retrieval and reinforcement learning. The source code is released at <https://github.com/KaiyangZhou/mixstyle-release>.

1 Introduction

Convolutional neural networks (CNNs) have been a key driving force behind successes in computer vision over the past decade (Krizhevsky et al., 2012; He et al., 2016; Szegedy et al., 2015). For example, given millions of labeled images for training, CNNs can surpass humans in classifying images into 1,000 categories on ImageNet (He et al., 2016). In reinforcement learning (RL), CNNs have also shown remarkable ability in extracting meaningful representations enabling RL

agents to play Atari games at near-human level (Mnih et al., 2013). However, these achievements are largely limited to independent and identically distributed (i.i.d.) datasets. When it comes to out-of-distribution (OOD) target datasets, which are common in practice, CNNs often fail catastrophically with low performance that is far away from that seen in source datasets (Zhou et al., 2021). This significantly hinders their deployment in real-world applications.

To mitigate domain discrepancy between training and test data, a straightforward solution is to collect diverse source data from multiple relevant domains so a CNN model is allowed to intrinsically learn more domain-invariant, and hence generalizable representations. This has long been studied under domain generalization (DG) (Zhou et al., 2021; Blanchard et al., 2011; Muandet et al., 2013; Fan et al., 2022). However, DG models in practice are often constrained by the amount (and diversity) of source domains. The reason is clear: collecting more training data with annotation is expensive and time-consuming; this is further compounded when the data has to come from multiple and diverse domains. Recent studies have shown promising results with neural networks-based data augmentation methods, such as perturbing images with adversarial gradients (Shankar et al., 2018) or learning neural networks to generate novel-looking images (Zhou et al., 2020). Such a model-based data augmentation (Zhou et al., 2021) approach is, however, less appealing

Communicated by Oliver Zendel.

✉ Kaiyang Zhou
kyzhou@hkbu.edu.hk
Yongxin Yang
yongxin.yang@qmul.ac.uk
Yu Qiao
yu.qiao@siat.ac.cn
Tao Xiang
t.xiang@surrey.ac.uk

- ¹ Hong Kong Baptist University, Hong Kong SAR, China
- ² Queen Mary University of London, London, UK
- ³ Shanghai AI Laboratory & Shenzhen Institute of Advanced Technology, CAS, Shenzhen, China
- ⁴ University of Surrey, Guildford, UK

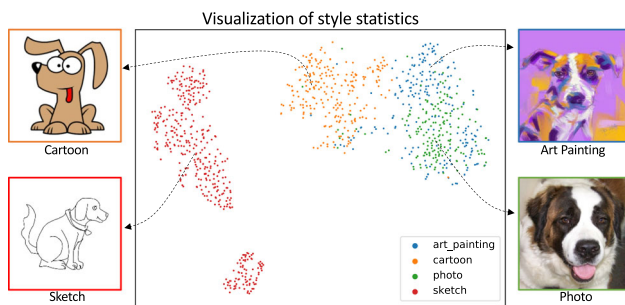


Fig. 1 2D t-SNE (Maaten & Hinton, 2008) visualization of style statistics (concatenation of means and standard deviations) computed from the first residual block's feature maps of a ResNet-18 (He et al., 2016) trained on four distinct domains (Li et al., 2017). The idea of our MixStyle approach is to mix these instance-level feature statistics to efficiently synthesize novel domains

to real-world deployment because it often consumes considerable compute resources in terms of GPU memories and training hours, as well as considerable engineering efforts.

In this work, we propose *MixStyle*, a plug-and-play, parameter-free module that can be simply inserted to shallow CNN layers to achieve implicit data augmentation and requires no modification to training objectives. Specifically, MixStyle probabilistically mixes the feature statistics—i.e., means and standard deviations of feature maps—between random instances. The idea is inspired by the observation that visual domains can often be characterized by image styles, which are in turn encapsulated within instance-level feature statistics in shallow CNN layers (Huang & Belongie, 2017; Dumoulin et al., 2017). An example is shown in Fig. 1: the four images depict the same semantic concept, i.e. dog, but have distinctive styles (e.g., characteristics in colors and textures); and the feature statistics clearly capture these styles reflected in the separable clusters. Therefore, MixStyle essentially synthesizes novel domains. Though designed for tackling the DG problem, MixStyle can also be applied for problems where unlabeled images are available, e.g., semi-supervised domain generalization (Zhou et al., 2021) and unsupervised domain adaptation (Ganin & Lempitsky, 2015). This is achieved by introducing a simple extension by mixing feature statistics between labeled and pseudo-labeled instances. It is worth noting that MixStyle i) is architecture-agnostic, ii) does not necessarily require domain labels as combining instance styles can also lead to new styles, iii) perfectly fits into mini-batch training, and iv) is easy to implement with only a few lines of code.

The contributions of this paper are summarized as follows. First, to facilitate data augmentation for domain generalization, we for the first time introduce a new concept called MixStyle, which builds a connection between visual domains and feature statistics that control image styles. Second, we provide an efficient implementation of MixStyle through

simple manipulation of feature statistics. Third, a MixStyle-based semi-supervised learning framework is proposed to extend the applicability of MixStyle to partially labeled datasets. Finally, extensive experiments are conducted to demonstrate the effectiveness and versatility of MixStyle in the problems of domain generalization, semi-supervised domain generalization, and unsupervised domain adaptation, and covering various tasks including object recognition, instance retrieval, and RL. Comprehensive ablation studies are also provided to give an in-depth understanding into the mechanism of MixStyle, as well as to share insights on how to apply MixStyle in practice. All source code of this work has been made publicly available to facilitate future research.¹

An earlier and preliminary study of MixStyle was published in ICLR 2021 (Zhou et al., 2021). In comparison, this paper introduces substantial new materials: 1) MixStyle can now work with unlabeled data via a non-trivial design that allows the mixing of feature statistics between labeled and pseudo-labeled instances; 2) We demonstrate that MixStyle works exceptionally well with extremely limited labels like 5 labels per category; 3) Extensive experiments covering semi-supervised domain generalization (Zhou et al., 2021) and unsupervised domain adaptation (Ganin & Lempitsky, 2015) are conducted.

2 Related Work

Domain Generalization (DG) studies the problem of model generalization to out-of-distribution data using only source data that are gathered from multiple relevant domains. We refer readers to a recent survey paper (Zhou et al., 2021) for a more comprehensive literature review in this topic. Numerous methods are based on the idea of aligning feature distributions across different source domains. The so-called domain alignment methods often minimize a distance metric that quantifies the source domain discrepancy, such as those based on maximum mean discrepancy (MMD) (Li et al., 2018b), contrastive losses (Motiian et al., 2017), and adversarial learning (Li et al., 2018c). Since source data cover multiple distinct domains, ensemble learning has also been explored where the main idea is to learn domain-specific models, such as domain-specific classifiers (Zhou et al., 2020b; Ding & Fu, 2017) or batch normalization layers (Seo et al., 2020; Segu et al., 2023), and use their ensemble for prediction. Recently, meta-learning has drawn increasing attention from the DG community (Li et al., 2018a; Balaji et al., 2018; Dou et al., 2019). The key idea is to construct training episodes that are fed to the model at each forward-backward step. Each episode contains a pseudo-train and a pseudo-test set with non-overlapping domains, both derived

¹ <https://github.com/KaiyangZhou/mixstyle-release>

from source domains. The model is typically optimized in the pseudo-train set in a way that the performance in the pseudo-test set is improved, which often requires second-order differentiation.

More related to our work are data augmentation methods. CrossGrad (Shankar et al., 2018) perturbs the input to a category classifier with adversarial gradients back-propagated from a domain classifier. DDAIG (Zhou et al., 2020a) learns a perturbation neural network to synthesize images that cannot be recognized by a domain classifier. L2A-OT (Zhou et al., 2020) learns a neural network to map source data to pseudo-novel domains by maximizing an optimal transport-based distance measure. The proposed MixStyle is related to L2A-OT in its effort to synthesizing novel domains. However, MixStyle differs from L2A-OT in its much simpler formulation that leverages feature statistics, as well as its more efficient implementation that requires only a few lines of code. Essentially, MixStyle can be seen as feature-level augmentation, which is clearly different from image-level augmentation methods like L2A-OT.

Generalization in Deep RL Reinforcement learning (RL) agents often overfit training environments and as a result perform poorly in unseen environments with different visual patterns or difficulty levels (Zhang et al., 2018a). A natural way to improve generalization, which has been shown effective in several studies (Cobbe et al., 2019; Farebrother et al., 2018), is to impose regularization, such as weight decay. However, Igl et al. (2019) suggest that stochastic regularization methods like dropout (Srivastava et al., 2014) and batch normalization (Ioffe & Szegedy, 2015)—the latter relies on estimated population statistics—can cause some adverse effects because the training data in RL are essentially model-dependent. They propose selective noise injection (SNI) that combines a stochastic regularization method with its deterministic counterpart. They further integrate SNI with information bottleneck actor critic (IBAC-SNI) to reduce the variance in gradients. Justesen et al. (2018) design a curriculum learning scheme where the level of training episodes progresses from easy to difficult during the course of training. Advances in image-to-image translation have also been exploited, e.g., Gamrian and Goldberg (2019) use an image translation model to map target data to the source domain where the agent was trained on to solve the domain shift problem. Tobin et al. (2017) introduce domain randomization, which diversifies training data by rendering images with different visual effects via a programmable simulator. Lee et al. (2020) pre-process input images with a randomly initialized network for data augmentation. A couple of recent studies (Laskin et al., 2020; Kostrikov et al., 2021) have shown that it is useful to combine a diverse set of label-preserving transformations, such as rotation, shifting and Cutout (DeVries & Taylor, 2017). MixStyle is related to the idea of domain randomization but does not need specialized

simulators to achieve domain augmentation. MixStyle works efficiently at feature level and is orthogonal to most existing methods, such as IBAC-SNI (see Section 4.1.3), as it can be simply plugged into RL agents' CNN backbone.

Unsupervised Domain Adaptation (UDA) is closely related to DG because both aim to overcome the domain shift problem encountered in a new target domain. The biggest difference lies in whether target data are available for training: the former assumes unlabeled target data are available whereas the latter relies purely on source data. A large body of UDA methods (Ganin & Lempitsky, 2015; Sun & Saenko, 2016; Tzeng et al., 2017; Long et al., 2016; Deng et al., 2019; Kang et al., 2020; Saito et al., 2018; Liu et al., 2020) fall into the group of domain alignment, which sounds familiar in the DG literature. However, in UDA this idea is backed by a theoretical guarantee (Ben-David et al., 2010) that reducing the distribution mismatch between the source and target domain can lead to a lower upper bound on the target error. Methods fallen into this group often seek to minimize an explicit distance function based on, e.g., moments (Sun & Saenko, 2016) or MMD (Long et al., 2016), or learn a domain discriminator in an adversarial manner that equivalently minimizes the Jensen-Shannon divergence (Ganin & Lempitsky, 2015; Tzeng et al., 2017). Recent studies (Deng et al., 2019; Saito et al., 2018; Kang et al., 2020) have suggested that fine-grained, class-specific alignment outperforms coarse alignment—the latter might result in misalignment in samples of different classes. Besides feature-level alignment, pixel-level alignment has also been extensively researched where the key idea is to transform source images into the target style so a target classifier can be directly trained (Hoffman et al., 2018; Gong et al., 2019). A related work is DLOW (Gong et al., 2019), which maps source images to intermediate domains as well as the target domain to achieve model adaptation for semantic segmentation. When applied to UDA, our MixStyle mixes source and target feature statistics, which can be seen as generating intermediate domains, like DLOW—but in an implicit yet much more efficient way.

3 MixStyle Neural Networks

Generalization to unseen domains using only source data for training is a challenging task. Our motivation to improve out-of-distribution generalization for neural networks is to diversify the source data distributions (domains) as much as possible. This allows the model to discover more generalizable patterns by itself through learning. However, manually collecting data from more sources for data augmentation is both expensive and time-consuming. To achieve data augmentation in a more efficient way, we resort to feature-level augmentation.

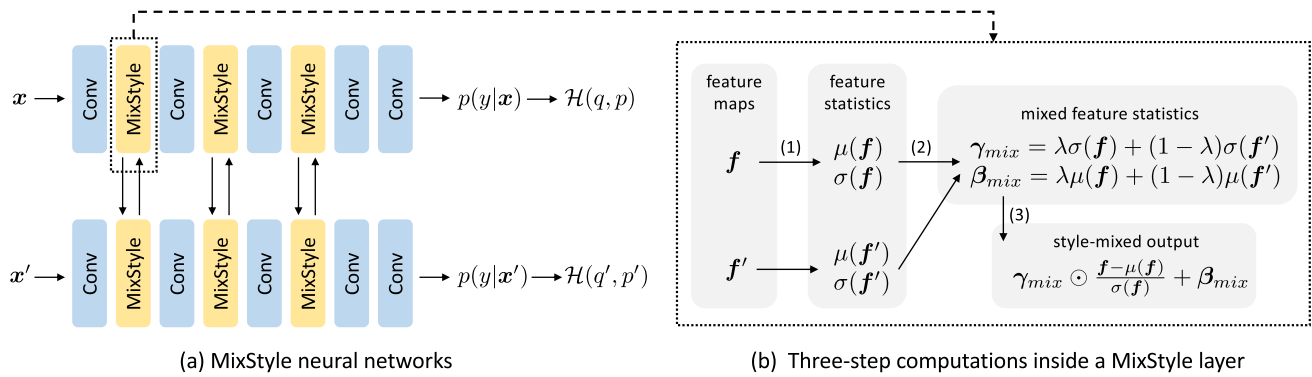


Fig. 2 A schematic overview of MixStyle neural networks. The best practice is to insert MixStyle to multiple shallow layers. x and x' denote two image instances, with q and q' being their ground-truth labels (or one of them being pseudo label if dealing with unlabeled data). $\mathcal{H}(\cdot, \cdot)$ means the cross-entropy loss. f and f' are feature maps extracted at a

Our proposed approach, *MixStyle*, is a plug-and-play CNN module that probabilistically mixes the CNN feature statistics between instances to implicitly synthesize “new” styles. This is inspired by recent studies (Ulyanov et al., 2016; Huang & Belongie, 2017; Dumoulin et al., 2017) showing that CNN feature statistics—specifically the channel-wise means and standard deviations of feature maps—essentially capture the style of an image, which can be viewed as a characterization of visual domains.

A schematic overview of MixStyle neural networks is shown in Fig. 2. It is worth noting that our MixStyle module is totally parameter-free and does not need to store any buffer, hence being highly efficient, as well as easy to implement—it only requires a few lines of code using existing open-source deep learning frameworks like PyTorch or TensorFlow. Moreover, there is no need to modify the learning objectives. One can follow the original training paradigm, such as using the cross-entropy loss for image classification tasks or policy gradient methods for reinforcement learning.

Below we introduce in more detail the background on CNN feature statistics, the design of MixStyle module, and finally how MixStyle can be extended to cope with unlabeled data for semi-supervised learning.

3.1 Background

Instance Normalization Most recent style transfer models are based on feedforward neural networks with an encoder-decoder architecture (Huang & Belongie, 2017; Dumoulin et al., 2017). Ulyanov et al. (2016) identify that replacing batch normalization in feedforward style transfer models with instance normalization can effectively remove instance-specific styles, facilitating image style transfer. The basic idea of instance normalization is to normalize feature maps with means and standard deviations computed across the spa-

tial dimension within each feature channel, followed by a learnable affine transformation. Formally, let $f \in \mathbb{R}^{C \times H \times W}$ be feature maps for an image (extracted at a certain CNN layer), with C , H and W denoting the dimension of channel, height and width, respectively, instance normalization (IN) is defined as

$$\text{IN}(f) = \gamma \odot \frac{f - \mu(f)}{\sigma(f)} + \beta, \quad (1)$$

where $\gamma, \beta \in \mathbb{R}^C$ denote the affine transformation parameters; $\mu(f), \sigma(f) \in \mathbb{R}^C$ are the means and standard deviations computed within each channel of f . Specifically, for $c \in \{1, \dots, C\}$,

$$\mu(f)_c = \frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W f_{c,h,w}, \quad (2)$$

and

$$\sigma(f)_c = \sqrt{\frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W (f_{c,h,w} - \mu(f)_c)^2}. \quad (3)$$

Adaptive Instance Normalization Since the feature statistics are linked to image style, Huang and Belongie (2017) propose adaptive instance normalization (AdaIN) that replaces the feature statistics of a content image with those of a style image to achieve arbitrary image style transfer. Specifically, AdaIN replaces the affine transformation parameters, γ and β in Eq. (1), with the feature statistics of a style image, $\mu(f')$ and $\sigma(f')$, where f' denotes the style image’s feature maps,

$$\text{AdaIN}(f, f') = \sigma(f') \odot \frac{f - \mu(f)}{\sigma(f)} + \mu(f'). \quad (4)$$

3.2 MixStyle Module

MixStyle draws inspiration from AdaIN. However, rather than attaching a decoder for image generation, MixStyle is designed for the purpose of regularizing CNN training by perturbing the style information of source domain instances. MixStyle is inserted between layers (blocks) in a CNN architecture, as shown in Fig. 2a.

More specifically, MixStyle mixes the feature statistics of two instances with a random convex weight. The computations inside a MixStyle module can be summarized into three steps (depicted in Fig. 2b). First, given two sets of feature maps $\mathbf{f}$ and $\mathbf{f}'$ for two instances, MixStyle computes their feature statistics, $(\mu(\mathbf{f}), \sigma(\mathbf{f}))$ and $(\mu(\mathbf{f}'), \sigma(\mathbf{f}'))$. Second, MixStyle generates a mixture of feature statistics,

$$\gamma_{mix} = \lambda \sigma(\mathbf{f}) + (1 - \lambda) \sigma(\mathbf{f}'), \quad (5)$$

$$\beta_{mix} = \lambda \mu(\mathbf{f}) + (1 - \lambda) \mu(\mathbf{f}'), \quad (6)$$

where λ is an instance-specific, random weight sampled from the beta distribution, $\lambda \sim \text{Beta}(\alpha, \alpha)$ with $\alpha \in (0, \infty)$ being a hyper-parameter. We suggest setting $\alpha = 0.1$ in practice.

Finally, the mixture of feature statistics is applied to the style-normalized $\mathbf{f}$,

$$\text{MixStyle}(\mathbf{f}, \mathbf{f}') = \gamma_{mix} \odot \frac{\mathbf{f} - \mu(\mathbf{f})}{\sigma(\mathbf{f})} + \beta_{mix}. \quad (7)$$

Essentially, AdaIN is a special case of MixStyle when $\lambda = 0$. We use a probability of 0.5 to decide if MixStyle is activated or not in the forward pass. MixStyle is *not* used at test time. Gradients are blocked in the computational graph of $\mu(\cdot)$ and $\sigma(\cdot)$ to prevent the augmentation effect from being erased.

Mini-Batch Training MixStyle can be easily integrated into mini-batch training. Specifically, $\text{MixStyle}(\mathbf{f}, \mathbf{f}')$ in Eq. (7) can be modified to $\text{MixStyle}(\mathbf{F}, \mathbf{F}')$, with $\mathbf{F}, \mathbf{F}' \in \mathbb{R}^{B \times C \times H \times W}$ being two batches of feature maps (B denotes the batch size). A simple realization is to shuffle the order in the batch dimension of $\mathbf{F}$ to obtain $\mathbf{F}'$. When source domain labels are available, one can also force style mixing to occur between images of different domains. This can be implemented by sampling half of the B images from one source domain while the other half from a different source domain, and switching the order of the two halves in $\mathbf{F}$ to produce $\mathbf{F}'$. Fig. 1 suggests that sub-domains exist within each domain, so even if two instances of the same domain are mixed, new domains could still be synthesized. We empirically find that the performance of random mixing is comparable to that of cross-domain mixing.

3.3 Extension to Semi-Supervised Learning

We present a simple MixStyle-based semi-supervised learning framework to deal with unlabeled data. This new framework can be applied to both semi-supervised domain generalization (Zhou et al., 2021), where the source data are only partially labeled, and unsupervised domain adaptation (Lu et al., 2020; Saito et al., 2018), where the goal is to adapt a model from a labeled source dataset to an unlabeled target dataset. Compared to supervised learning, the semi-supervised version mainly differs in that two to-be-mixed instances (or mini-batches) come from the labeled and the unlabeled dataset respectively. More specifically, $\mathbf{x}$ in Fig. 2a is a labeled instance while $\mathbf{x}'$ is now an unlabeled instance. For semi-supervised domain generalization, $\mathbf{x}'$ is sampled from the unlabeled source dataset, whereas for unsupervised domain adaptation, $\mathbf{x}'$ comes from the unlabeled target domain.

Since the MixStyle training requires proper supervision for learning meaningful representations, we propose to assign pseudo labels to the unlabeled data using predictions from the model itself. For each unlabeled instance $\mathbf{x}'$, the model produces a probability distribution over all categories, and the one with the highest confidence estimate is selected as the pseudo label. Following FixMatch (Sohn et al., 2020), a state-of-the-art pseudo-labeling method, we use a confidence threshold to filter out low-confidence predictions, and adopt the weak-strong augmentation training to reduce overfitting to noisy pseudo labels—pseudo labels are estimated on weakly augmented images while predictions are made on strongly augmented images.

Formally, given a batch of B instances—half of them are labeled and half of them are unlabeled—we have a supervised loss ℓ_s , which is computed on the labeled instances,

$$\ell_s = \sum_{i=1}^{\frac{B}{2}} \mathcal{H}(q_i, p(y|a(\mathbf{x}_i))), \quad (8)$$

where $\mathcal{H}(\cdot, \cdot)$ is the cross-entropy loss, q_i is the ground-truth label of $\mathbf{x}_i$, and $a(\cdot)$ denotes a weak augmentation function (e.g., random flip and crop).

We also have an unlabeled loss ℓ_u , which is computed on the unlabeled instances,

$$\ell_u = \sum_{i=1}^{\frac{B}{2}} \mathbb{1}(\max(p'_i) \geq \tau) \mathcal{H}(q'_i, p(y|A(\mathbf{x}'_i))), \quad (9)$$

where $\max(p'_i)$ is the most confident estimate, τ is the confidence threshold (fixed to 0.95), q'_i is the pseudo label obtained on the weakly augmented image $a(\mathbf{x}'_i)$, and $A(\cdot)$ denotes a

strong augmentation function (e.g., RandAugment (Cubuk et al., 2019)).

4 Experiments

4.1 Domain Generalization

To demonstrate the versatility of MixStyle, our experiments cover a wide range of generalization tasks: object recognition (Sect. 4.1.1), instance retrieval (Sect. 4.1.2), and reinforcement learning (Sect. 4.1.3).

4.1.1 Object Recognition

Experimental Setup We conduct experiments on two commonly used domain generalization (DG) datasets, namely PACS (Li et al., 2017) and Office-Home (Venkateswara et al., 2017), which contain different types of domain shifts. PACS consists of four domains—art painting, cartoon, photo and sketch—with a total of 9991 images and seven classes. The domain shifts are mainly caused by image style changes (Fig. 3a). Office-Home also comprises four domains, which are art, clipart, product and real-world, but contains more images than PACS: around 15,500 images of 65 classes related to objects in office and home environments (Fig. 3b). Following previous work (Li et al., 2019; Zhou et al., 2020), we use the leave-one-domain-out protocol for evaluation. Specifically, we pick three domains as the source to train a model, which is then deployed in the remaining domain seen as the target. All experiments are repeated five times with different random seeds and the average results are reported.

Implementation Details We adopt the commonly used ImageNet-pretrained ResNet-18 (He et al., 2016) as the CNN backbone, same as existing methods (Carlucci et al., 2019; Li et al., 2019; Zhou et al., 2020). We set the batch size to 64 and train the model using the SGD optimizer for 50 epochs. The initial learning rate is set to 0.001 and decayed by the cosine annealing rule (Loshchilov & Hutter, 2017). For PACS, MixStyle is inserted after the 1st, 2nd and 3rd residual blocks. For Office-Home, MixStyle is inserted after

the 1st and 2nd residual blocks. The study of where to insert MixStyle in a CNN will later be presented in Section 4.4. Our code is built using the public `Dassl.pytorch` toolbox (Zhou et al., 2020b).²

Baselines Our main competitors are general-purpose regularization methods including Mixup (Zhang et al., 2018b), Manifold Mixup (Verma et al., 2019), DropBlock (Ghiasi et al., 2018), CutMix (Yun et al., 2019), and Cutout (DeVries & Taylor, 2017). They are trained using the same optimization parameters as MixStyle, along with the optimal hyper-parameters setup as suggested in their papers. We also compare with the existing DG methods, which reported state-of-the-art (SOTA) results on PACS and Office-Home. These methods include the domain alignment-based CCSA (Motiian et al., 2017) and MMD-AAE (Li et al., 2018b), the Jigsaw puzzle-based JiGen (Carlucci et al., 2019), the adversarial gradient-based CrossGrad (Shankar et al., 2018), the meta-learning-based Metareg (Balaji et al., 2018) and Epi-FCR (Li et al., 2019), and the data augmentation-based L2A-OT (Zhou et al., 2020).

Results The comprehensive results are presented in Table 1. We first discuss the comparisons with the general-purpose regularization methods. Overall, we observe that the general-purpose regularization methods do not offer any clear advantage over the vanilla ResNet-18 in these DG tasks. In contrast, MixStyle (both the random and cross-domain mixing versions) consistently improves upon the vanilla model on both datasets. Compared with Mixup, which mixes images in the pixel space, our feature statistics-based MixStyle is around 4% better on PACS and 1% better on Office-Home. MixStyle also beats Manifold Mixup, which further justifies the advantage of mixing feature statistics for DG. MixStyle and DropBlock share some commonalities in that both are applied to feature maps at multiple layers, but MixStyle shows clear advantages over DropBlock in all domains. The reason why DropBlock is ineffective here is because dropping out feature activations mainly encourages the network to mine discriminative patterns but does not reinforce the ability to cope with unseen domains (styles), which is however what MixStyle aims to achieve through synthesizing “new” styles for data augmentation. In addition, it is interesting to see that the random mixing version of MixStyle is highly comparable to the cross-domain mixing version—the former achieves slightly better performance on two domains of PACS (cartoon and photo) and all domains of Office-Home. These results suggest that there indeed exist sub-domains within a single source domain, which allow random mixing to produce more diverse domains, and hence a more domain-generalizable model.

When it comes to SOTA DG methods, MixStyle outperforms most of them by a clear margin, despite having a



Fig. 3 Example images from PACS, Office-Home and the person re-ID benchmark

² <https://github.com/KaiyangZhou/Dassl.pytorch>

Table 1 Domain generalization results on PACS and Office-Home

Model	PACS				Office-Home					
	Art painting	Cartoon	Photo	Sketch	Avg	Art	Clipart	Product	Real-world	Avg
MMD-AAE Li et al. (2018b)	75.2	72.7	96.0	64.2	77.0	56.5	47.3	72.1	74.8	62.7
CCSA Motiian et al. (2017)	80.5	76.9	93.6	66.8	79.4	59.9	49.9	74.1	75.7	64.9
JiGen Carlucci et al. (2019)	79.4	75.3	96.0	71.6	80.5	53.0	47.5	71.5	72.8	61.2
CrossGrad Shankar et al. (2018)	79.8	76.8	96.0	70.2	80.7	58.4	49.4	73.9	75.8	64.4
Epi-FCR Li et al. (2019)	82.1	77.0	93.9	73.0	81.5	–	–	–	–	–
Metareg Balaji et al. (2018)	83.7	77.2	95.5	70.3	81.7	–	–	–	–	–
L2A-OT Zhou et al. (2020)	83.3	78.2	96.2	73.6	82.8	60.6	50.1	74.8	77.0	65.6
ResNet-18	77.0	75.9	96.0	69.2	79.5	58.9	49.4	74.3	76.2	64.7
+ Manifold Mixup Verma et al. (2019)	75.6	70.1	93.5	65.4	76.2	56.2	46.3	73.6	75.2	62.8
+ Cutout DeVries and Taylor (2017)	74.9	74.9	95.9	67.7	78.3	57.8	48.1	73.9	75.8	63.9
+ CutMix Yun et al. (2019)	74.6	71.8	95.6	65.3	76.8	57.9	48.3	74.5	75.6	64.1
+ Mixup Zhang et al. (2018b)	76.8	74.9	95.8	66.6	78.5	58.2	49.3	74.7	76.1	64.6
+ DropBlock Ghiasi et al. (2018)	76.4	75.4	95.9	69.0	79.2	58.0	48.1	74.3	75.9	64.1
+ MixStyle (random)	82.3	79.0	96.3	73.8	82.8	58.7	53.4	74.2	75.9	65.5
+ MixStyle (xdomain)	84.1	78.8	96.1	75.9	83.7	57.3	52.9	73.5	75.3	64.8

Bold values indicate the best result

MixStyle outperforms other strong regularization methods, such as DropBlock and CutMix, by a clear margin. MixStyle also achieves comparable performance with the main DG competitor, L2A-OT, which synthesizes novel-domain data in the input space—MixStyle achieves implicit data augmentation in the feature space, which is much more efficient

Table 2 Results on DomainBed

	PACS	VLCS	Office-Home
ERM	84.2	77.3	67.6
MixStyle	85.2	77.9	60.4

(Model selection: training-domain validation set.)

much simpler design that adds negligible overhead to a supervised classifier. On PACS, the cross-domain mixing version of MixStyle even surpasses the recently introduced L2A-OT, with nearly 1% improvement on average. From a data augmentation perspective, MixStyle and L2A-OT share a similar goal: to synthesize data from pseudo-novel domains. However, the ways they achieve the goal are distinguishable: MixStyle mixes feature statistics in each forward pass, which is highly efficient; whereas L2A-OT trains a deep image generation network by iteratively maximizing the domain difference measured by optimal transport between the original and generated images. The latter introduces much heavier computational cost than MixStyle in both GPU memories and training time, and also requires more engineering effort.

DomainBed We also conduct experiments on the DomainBed benchmark (Gulrajani & Lopez-Paz, 2020) where a larger model is used, i.e., ResNet50, and each method undergoes an extensive parameter tuning process. We compare MixStyle with the ERM model and show the results on Table 2. Overall, we observe that with the model becoming larger, MixStyle's advantage over ERM shrinks, which is similar to other DG methods (Gulrajani & Lopez-Paz, 2020).

On PACS and VLCS, MixStyle beats ERM. But on Office-Home, MixStyle underperforms ERM by a large margin.

4.1.2 Instance Retrieval

Experimental Setup We evaluate MixStyle on the person re-identification (re-ID) task, which aims to match people across disjoint camera views. As each camera view is itself a distinct domain, person re-ID is essentially a cross-domain image (instance) matching problem. We select two datasets for evaluation, namely Market1501 (Zheng et al., 2015) and Duke (Ristani et al., 2016; Zheng et al., 2017). Market1501 contains 32,668 images of 1501 identities captured by 6 cameras, while Duke contains 36,411 images of 1,812 identities captured by 8 cameras. See Fig. 3c for image examples. A model is trained using one dataset and tested on the other.³ Images from these two datasets manifest substantial differences in terms of backgrounds, illuminations, image resolutions, viewpoints, and so on, which greatly challenge cross-domain recognition. Ranking accuracy and mean average precision (mAP) (Zheng et al., 2015) are used as the performance measures.

Implementation Details We evaluate MixStyle using two different CNN architectures: ResNet-50 (He et al., 2016) and OSNet (Zhou et al., 2021). The latter was specifically designed for re-ID. In both architectures, MixStyle is inserted

³ We follow the original train/test split in each dataset for evaluation.

Table 3 Results on the cross-dataset person re-identification (re-ID) benchmark

Model	Market1501→Duke				Duke→Market1501			
	mAP	R1	R5	R10	mAP	R1	R5	R10
ResNet-50	19.3	35.4	50.3	56.4	20.4	45.2	63.6	70.9
+ RandomErase Zhong et al. (2020)	14.3	27.8	42.6	49.1	16.1	38.5	56.8	64.5
+ DropBlock Ghiasi et al. (2018)	18.2	33.2	49.1	56.3	19.7	45.3	62.1	69.1
+ MixStyle (random)	23.8	42.2	58.8	64.8	24.1	51.5	69.4	76.2
+ MixStyle (xdomain)	23.4	43.3	58.9	64.7	24.7	53.0	70.9	77.8
OSNet	25.9	44.7	59.6	65.4	24.0	52.2	67.5	74.7
+ RandomErase Zhong et al. (2020)	20.5	36.2	52.3	59.3	22.4	49.1	66.1	73.0
+ DropBlock Ghiasi et al. (2018)	23.1	41.5	56.5	62.5	21.7	48.2	65.4	71.3
+ MixStyle (random)	27.2	48.2	62.7	68.4	27.8	58.1	74.0	81.0
+ MixStyle (xdomain)	27.3	47.5	62.0	67.1	29.0	58.2	74.9	80.9

Bold values indicate the best result

MixStyle demonstrates clear advantages over the baseline methods in all settings. It is worth noting that the domain shift problem here is much more challenging than that in the object recognition benchmarks since the person images are captured by real-world cameras deployed in complex scenes

after the 1st and 2nd residual blocks. Following Zhou et al. (2021), we train the model to perform classification on training identities and use features extracted from the penultimate layer (before the linear classifier) for image matching. SGD is used as the optimizer. The batch size is set to 32 and the total number of epochs is 60. The global learning rate is set to 0.05 and decayed by 0.1 every 20 epochs. A smaller learning rate ($0.2 \times$ the global learning rate) is applied to the pretrained layers. Our code is built using the Torchreid library (Zhou et al., 2019).⁴

Baselines We compare with three baseline methods: 1) The vanilla model, which serves as a strong baseline; 2) DropBlock, which was the top-performing competitor in the object recognition experiment; 3) RandomErase (Zhong et al., 2020), a widely used regularization method in the re-ID literature (similar to Cutout).

Results The results are reported in Table 3. It is encouraging that MixStyle consistently outperforms the strong vanilla model under both settings and with different architectures. This strongly demonstrates the generality of MixStyle. Again, the results of the random and cross-domain mixing are generally comparable to each other, suggesting that domain labels are indeed not a must. In contrast, DropBlock and RandomErase are unable to show any benefit. Notably, RandomErase, which simulates occlusion by erasing pixels in random rectangular regions with random values, has been used as a default trick when training re-ID models. However, RandomErase shows a detrimental effect in the cross-dataset setting, suggesting that a further investigation into the use of data augmentation methods for practical re-ID is required. Similar to DropBlock, randomly erasing pixels offers no

guarantee to improve the robustness when it comes to domain shifts.

4.1.3 Reinforcement Learning

Experimental Setup We conduct experiments on the Coin-run benchmark (Cobbe et al., 2019), which is a recently introduced RL dataset particularly for evaluating the generalization performance of RL algorithms. As shown in Fig. 4a, the goal is to control the character to collect golden coins while avoiding both stationary and dynamic obstacles. Training data are sampled from 500 levels whereas test data are sampled from new levels of only the highest difficulty.

Implementation Details We follow Igl et al. (2019) to construct and train the RL agent. The CNN architecture used by IMPALA (Espeholt et al., 2018) is adopted as the policy network and trained by the proximal policy optimization algorithm (Schulman et al., 2017). We refer readers to Igl et al. (2019) for more implementation details. MixStyle is inserted after the 1st and 2nd convolutional sequences. As domain labels are difficult to define, we only evaluate the random mixing version of MixStyle. Our code is built on top of Igl et al. (2019).⁵

Baselines Following Igl et al. (2019), we train strong baseline models and add MixStyle on top of them to see whether MixStyle can bring further improvements. To this end, we train two baseline models: 1) Baseline, which combines weight decay and data augmentation;⁶ and 2) IBAC-SNI (the $\lambda = 0.5$ version) (Igl et al., 2019), which is a SOTA method based on selective noise injection.

⁵ <https://github.com/microsoft/IBAC-SNI>

⁶ We do not use batch normalization or dropout because they are detrimental to the performance, as suggested by Igl et al. (Igl et al., 2019).

⁴ <https://github.com/KaiyangZhou/deep-person-reid>

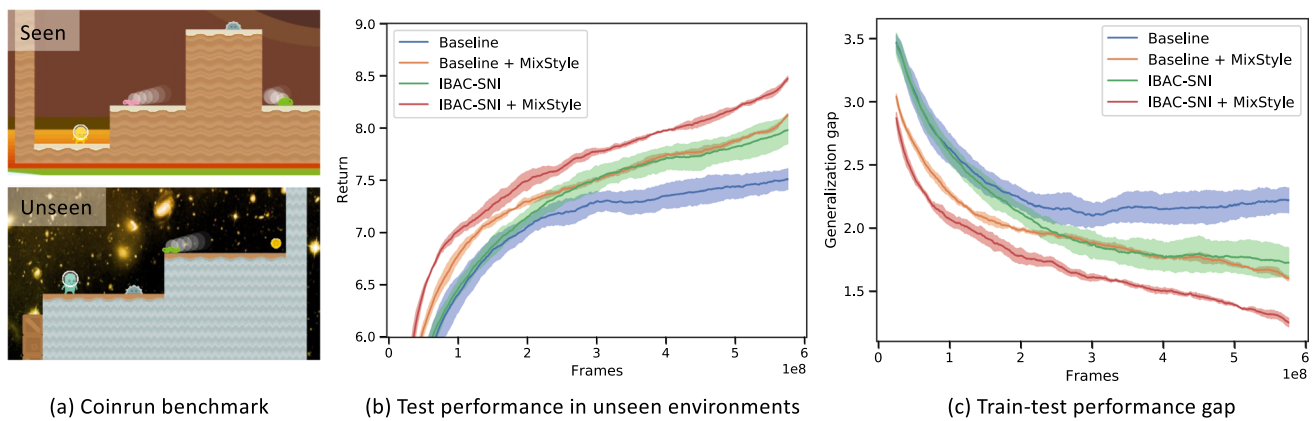


Fig. 4 Generalization results on the Coinrun benchmark. MixStyle significantly boosts the generalization performance of both the baseline model and the state-of-the-art method, IBAC-SNI

Results The test performance in unseen environments is summarized in Fig. 4b. Comparing Baseline with Baseline+MixStyle, we can see that MixStyle brings a remarkable improvement. Interestingly, MixStyle also significantly reduces the variance, as indicated by the smaller shaded area (for both orange and red lines). These results strongly demonstrate the effectiveness of MixStyle in enhancing generalization for RL agents. When it comes to the stronger baseline IBAC-SNI, IBAC-SNI+MixStyle is able to further boost the performance, suggesting that MixStyle is complementary to IBAC-SNI. This result also shows the potential of MixStyle as a plug-and-play component to be combined with other advanced RL methods. It is worth noting that Baseline+MixStyle itself is already highly competitive with IBAC-SNI. Fig. 4c plots the generalization gap, from which we observe that the models trained with MixStyle (orange & red lines) clearly generalize faster and better than those without using MixStyle (blue & green lines).

4.2 Semi-Supervised Domain Generalization

Semi-supervised domain generalization (SSDG) (Zhou et al., 2021) considers a more practical scenario where the multi-domain source data are partially labeled. We demonstrate that MixStyle on its own can significantly boost the out-of-distribution generalization performance in the low-data regime with only a few labeled instances per category; and its simple extension that mixes feature statistics between labeled and pseudo-labeled instances shows promising results that rival the current SOTA SSDG methods.

Experimental Setup We follow the benchmarks designed by Zhou et al. (2021). Specifically, we use PACS and Office-Home (see Section 4.1.1 for their statistics). Evaluation is performed on both the 10- and 5-labels-per-class settings (the rest are seen as unlabeled data) and over five random splits.

Implementation Details The ImageNet-pretrained ResNet-18 is used as the CNN backbone. The configurations for MixStyle are the same as in Section 4.1.1. When training with the labeled data only, we randomly sample a mini-batch of 32 images from a mixture of source domains. Whereas for the semi-supervised extension, we sample 16 images from each source domain to construct a mini-batch (for labeled and unlabeled data respectively). Other training details are kept the same as Zhou et al. (2021) for fair comparison.⁷

Baselines Similar to the DG experiment on object recognition (Section 4.1.1), we compare with the selected general-purpose regularizers. We also copy the results of other DG and SSDG methods from Zhou et al. (2021) for comparison. These include 1) the data augmentation-based CrossGrad (Shankar et al., 2018) and DDAIG (Zhou et al., 2020a), which can only be trained with fully labeled data. 2) the Jigsaw puzzle-based EISNet (Wang et al., 2020), and 3) the latest SSDG method StyleMatch (Zhou et al., 2021), which is based on a stochastic classifier and image-level style transfer for data augmentation. FixMatch (Sohn et al., 2020) is also compared here as an ablation study for our semi-supervised MixStyle.

Results The results on PACS and Office-Home are shown in Tables 4 and 5 respectively. We first discuss the comparisons of methods that only use the labeled source data for training. Compared with the vanilla training, MixStyle brings remarkable improvements on PACS, with over 6% increase given only 10 labels per class and nearly 7% increase in the much more challenging 5-labels-per-class setting, both measured by the average accuracy. On Office-Home, the gains are not as large as those on PACS but are still notable given that this dataset poses more challenges due to more classes and cluttered images (Zhou et al., 2021) and that most other methods struggle to beat the vanilla training. We also observe that the general-purpose regularizers and the two

⁷ <https://github.com/KaiyangZhou/ssdg-benchmark>

Table 4 Domain generalization results in the low-data regime on PACS, averaged over 5 random splits

Model	# labels: 210 (10 per class)					# labels: 105 (5 per class)				
	A	C	P	S	Avg	A	C	P	S	Avg
Vanilla	63.09	58.49	86.56	45.56	63.42	56.71	53.87	71.87	36.92	54.84
Manifold Mixup Verma et al. (2019)	60.29	54.21	81.60	39.88	59.00	53.72	49.90	69.98	34.48	52.02
Cutout DeVries and Taylor (2017)	62.15	60.09	86.97	44.65	63.46	55.48	55.97	73.04	36.16	55.16
CutMix Yun et al. (2019)	59.32	57.28	83.27	44.07	60.98	55.38	53.49	70.83	38.42	54.53
Mixup Zhang et al. (2018b)	64.28	57.92	85.79	42.69	62.67	57.71	54.33	73.01	37.16	55.56
DropBlock Ghiasi et al. (2018)	61.82	59.01	86.91	48.57	64.08	55.79	53.24	73.59	37.96	55.15
CrossGrad Shankar et al. (2018)	62.56	58.92	85.81	44.11	62.85	56.39	55.11	72.61	38.08	55.55
DDAIG Zhou et al. (2020a)	61.95	58.74	84.44	47.48	63.15	55.09	52.31	70.53	38.89	54.20
MixStyle	71.11	64.04	88.99	54.62	69.69	62.00	58.40	80.43	43.58	61.10
*EISNet Wang et al. (2020)	66.84	61.33	89.16	51.38	67.18	62.08	54.75	80.66	42.68	60.04
*FixMatch Sohn et al. (2020)	78.01	68.93	87.79	73.75	77.12	77.30	68.67	80.49	73.32	74.94
*StyleMatch Zhou et al. (2021)	79.43	73.75	90.04	78.40	80.41	78.54	74.44	89.25	79.06	80.32
*MixStyle	83.89	75.36	92.48	73.05	81.19	80.93	73.18	90.48	70.71	78.82

Bold values indicate the best result

A Art painting, C Cartoon, P Photo, S Sketch. * means the method uses unlabeled source data for training

Table 5 Domain generalization results in the low-data regime on Office-Home, averaged over 5 random splits

Model	# labels: 1950 (10 per class)					# labels: 975 (5 per class)				
	A	C	P	R	Avg	A	C	P	R	Avg
Vanilla	50.11	43.50	65.11	69.65	57.09	45.76	39.97	60.04	63.77	52.38
Manifold Mixup Verma et al. (2019)	47.15	39.78	63.75	67.75	54.61	44.40	37.29	58.78	62.31	50.70
Cutout DeVries and Taylor (2017)	50.36	43.38	65.01	69.86	57.15	45.70	39.63	59.67	63.70	52.18
CutMix Yun et al. (2019)	48.95	41.48	64.05	68.31	55.70	45.32	38.40	59.00	62.49	51.30
Mixup Zhang et al. (2018b)	49.67	43.87	64.83	69.12	56.87	46.29	40.30	59.02	63.87	52.37
DropBlock Ghiasi et al. (2018)	50.03	42.72	64.89	69.45	56.77	45.53	39.56	59.72	63.73	52.13
CrossGrad Shankar et al. (2018)	50.32	43.27	65.16	69.49	57.06	45.68	40.04	59.95	64.09	52.44
DDAIG Zhou et al. (2020a)	49.60	42.52	63.54	67.89	55.89	45.73	38.82	59.52	63.37	51.86
MixStyle	49.79	47.12	64.18	68.42	57.38	46.51	43.59	59.66	63.30	53.26
*EISNet Wang et al. (2020)	51.16	43.33	64.72	68.36	56.89	47.32	40.07	59.33	62.59	52.33
*FixMatch Sohn et al. (2020)	50.36	49.70	63.93	67.56	57.89	48.98	47.46	60.70	64.36	55.38
*StyleMatch Zhou et al. (2021)	52.82	51.60	65.31	68.61	59.59	51.53	50.00	60.88	64.47	56.72
*MixStyle	52.44	49.61	65.01	69.84	59.22	49.25	48.04	60.76	64.71	55.69

Bold values indicate the best result

A Art, C Clipart, P Product, R Real-world. * means the method uses unlabeled source data for training

data augmentation-based competitors are hugely challenged by the limited labels and are outperformed by MixStyle in most scenarios. When it comes to the comparisons of methods able to utilize the unlabeled source data, MixStyle shows encouraging results that can even rival the latest StyleMatch, despite being much more efficient thanks to the use of feature statistics.

4.3 Unsupervised Domain Adaptation

We further evaluate MixStyle on unsupervised domain adaptation (UDA) where style mixing occurs between the labeled

source and pseudo-labeled target images. The experiments cover both single- and multi-source settings. Our goal is not to beat SOTA UDA methods, but to demonstrate that such a simple design based on MixStyle can bring non-trivial improvements.

Experimental Setup We choose a challenging UDA dataset, VisDA-17 (Peng et al., 2017), which focuses on knowledge transfer from synthetic to real images. VisDA-17 consists of 152,397 labeled images containing synthetic 3D models and 55,388 unlabeled images derived from MS COCO (Lin et al., 2014). There are 12 classes in total. We follow previous work (Saito et al., 2018; Lu et al., 2020) to report the mean

Table 6 Unsupervised domain adaptation results on VisDA-17 using ResNet-101

Model	Accuracy
Source-only Saito et al. (2018)	52.4
MMD Long et al. (2015)	61.1
DANN Ganin and Lempitsky (2015)	57.4
MCD Saito et al. (2018)	71.9
SWD Lee et al. (2019)	76.4
STAR Lu et al. (2020)	82.7
FixMatch Sohn et al. (2020)	77.1
MixStyle	80.0

Bold value indicates the best result

The purpose here is not to achieve state-of-the-art performance but to show that MixStyle can be integrated into strong baseline methods like FixMatch

accuracy over 12 classes. For the multi-source setting, we follow Wang et al. (2020) to conduct experiments on PACS (Li et al., 2017).

Implementation Details On VisDA-17, we use the ImageNet-pretrained ResNet-101 (He et al., 2016) as the CNN backbone and replace the original 1000-way linear classifier with three fully connected layers, following prior work (Saito et al., 2018; Lu et al., 2020). MixStyle is inserted after the 1st and 2nd residual blocks. The model is trained with SGD and a constant learning rate of 0.001 for 10 epochs (Saito et al., 2018). The implementation on PACS mostly follows Section 4.2 except that each source mini-batch contains images randomly sampled from the mixture of source domains.

Results Table 6 presents the results on VisDA-17.⁸ We observe that the pseudo-labeling-based FixMatch is already a strong UDA model, which easily surpasses most previous specifically designed methods, such as MCD and DANN. Compared with FixMatch, MixStyle gains nearly 3%, which justifies the importance of the role of cross-domain style mixing. Table 7 shows the results on the multi-source setting where we have similar observations.

4.4 Ablation Study and Analysis

In this section, we examine the design choices in MixStyle and share insights on how to apply MixStyle in practice.

Where To Apply MixStyle? We repeat the DG experiments for object recognition and instance retrieval using the ResNet architecture. Given that a standard ResNet model has four residual blocks denoted by *res1*–4, we train different models with MixStyle applied to different layers. For notation, *res1* means MixStyle is applied after the 1st residual block;

⁸ Note that for fair comparison we only select the baselines that share a similar implementation including the model architecture.

Table 7 Multi-source unsupervised domain adaptation results on PACS

Model	A	C	P	S	Avg
Source-only Wang et al. (2020)	75.97	73.34	91.65	64.23	76.30
MDAN Zhao et al. (2018)	83.54	82.34	92.91	72.42	82.80
DCTN Xu et al. (2018)	84.67	86.72	95.60	71.84	84.71
M ³ SDA Peng et al. (2019)	84.20	85.68	94.47	74.62	84.74
MDDA Zhao et al. (2020a)	86.73	86.24	93.89	77.56	86.11
LtC-MSDA Wang et al. (2020)	90.19	90.47	97.23	81.53	89.85
FixMatch Sohn et al. (2020)	90.67	91.56	98.83	80.15	90.30
MixStyle	90.77	91.05	99.42	88.94	92.55

Bold values indicate the best result

MixStyle improves upon the strong baseline model, FixMatch, by a clear margin

Table 8 Ablation study on where to apply MixStyle using the ResNet architecture as an example

Model	Accuracy
(a) Object recognition	
ResNet-18	79.5
+ MixStyle (<i>res1</i>)	80.1
+ MixStyle (<i>res12</i>)	81.6
+ MixStyle (<i>res123</i>)	82.8
+ MixStyle (<i>res1234</i>)	75.6
+ MixStyle (<i>res14</i>)	76.3
+ MixStyle (<i>res23</i>)	81.7
(b) Cross-dataset person re-ID	
ResNet-50	19.3
+ MixStyle (<i>res1</i>)	22.6
+ MixStyle (<i>res12</i>)	23.8
+ MixStyle (<i>res123</i>)	22.0
+ MixStyle (<i>res1234</i>)	10.2
+ MixStyle (<i>res14</i>)	11.1
+ MixStyle (<i>res23</i>)	20.6

Bold values indicate the best result

res12 means MixStyle is applied after both the 1st and 2nd residual blocks; and so forth. The results are shown in Table 8. We have the following observations. 1) *Applying MixStyle to multiple shallow layers generally achieves a better performance*—for instance, *res12* is better than *res1* on both tasks. 2) *Different tasks favor different combinations*—*res123* achieves the best performance on PACS, while on the re-ID datasets *res12* is the best. 3) On both tasks, *the performance plunges when applying MixStyle to the last residual block*. This makes sense because *res4* is the closest to the prediction layer and tends to capture semantic content (i.e. label-sensitive) information rather than

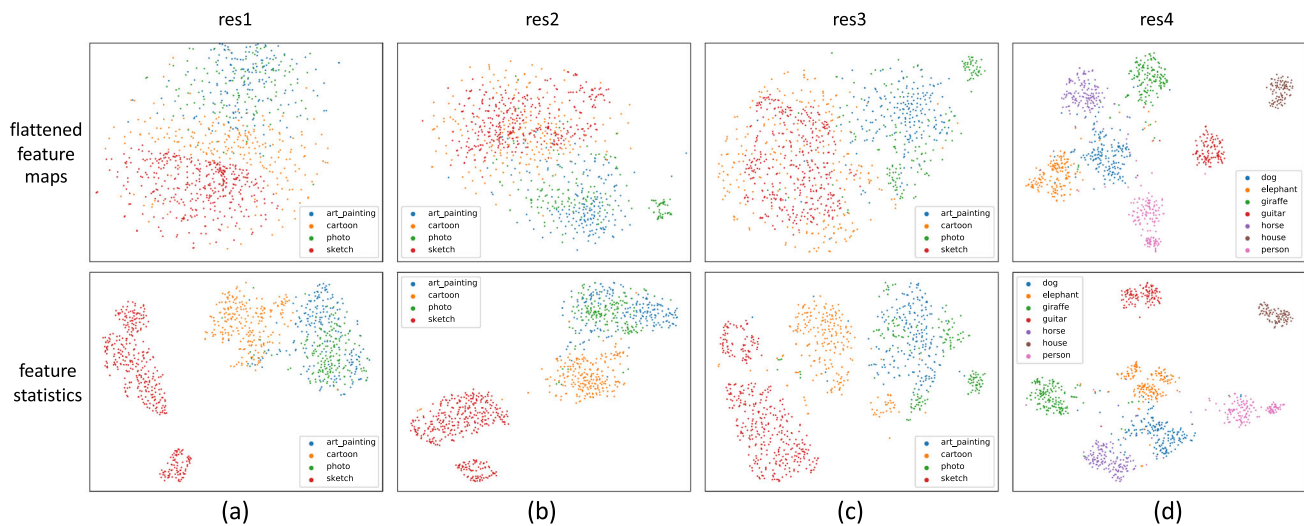


Fig. 5 2D visualization of flattened feature maps (top) and the corresponding style statistics (bottom). *res1–4* denote the four residual blocks in order in a ResNet architecture. We observe that *res1* to *res3* contain domain-related information while *res4* encodes label-related information

Table 9 Examining MixStyle’s design choices

Design	Accuracy
(a)	
Mixing	82.8 ±0.4
Replacing	82.1±0.5
(b)	
Random	82.8 ±0.4
Fixed	82.4±0.5
Bold values indicate the best result	

style. In particular, *res4* is followed by an average-pooling layer, which essentially forwards the mean vector to the prediction layer so it captures label-related information. As a consequence, mixing the statistics at *res4* breaks the inherent label space. This is clearer in Fig. 5: the features and style statistics in *res1–3* exhibit clustering patterns based on domains while those in *res4* have a high correlation with class labels.

Mixing vs. Replacing Unlike the AdaIN formulation that completely replaces one style with another, MixStyle gives a more general formulation that mixes two styles with a random convex combination. Table 9 suggests that mixing is a better choice—mixing produces more diverse styles (imagine an interpolation between two distant datapoints).

Random vs. Fixed Shuffle Table 8 confirms that applying MixStyle to multiple layers is better, but this may raise

another question over whether to shuffle the mini-batch at different layers or just follow the same shuffled order at all selected layers. Table 9b advocates the random shuffle design, probably due to the increased noise level that enhances regularization.

Random vs. Same-class Style Mixing We conduct an experiment to evaluate which style mixing strategy is better: style mixing among random samples vs. style mixing within the same class. The experiment is done on PACS and the results are shown on Table 10. It is clear that the “same-class” version’s performance is similar to the ERM model, which suggests that style mixing between samples of the same class is ineffective.

Beta Distribution’s Parameter In addition to positions to insert MixStyle, another hyper-parameter is α , the beta distribution’s shape parameter that controls the distribution of the convex weight λ in Eq. (7). A smaller α means λ is biased toward the extreme value of 0 or 1, while a larger α means λ is more likely to sit around 0.5. The study is summarized in Fig. 6. On PACS, the accuracy slides from 82.8% to 81.7% with α increasing from 0.1 to 0.4, but stabilizes thereafter. The results suggest that the performance is not too sensitive to α , and choosing $\alpha \in \{0.1, 0.2, 0.3\}$ seems to be a good start point. The results on the re-ID and RL task further confirm that the variance for different α ’s within $\{0.1, 0.2, 0.3\}$ is generally small. Overall, $\alpha = 0.1$ is a good default setting.

Table 10 Style mixing between random samples vs. samples of the same class

	Art painting	Cartoon	Photo	Sketch	Avg.
ERM	77.0	75.9	96.0	69.2	79.5
MixStyle (random)	82.3	79.0	96.3	73.8	82.8
MixStyle (same-class)	79.3	76.3	94.7	68.5	79.7

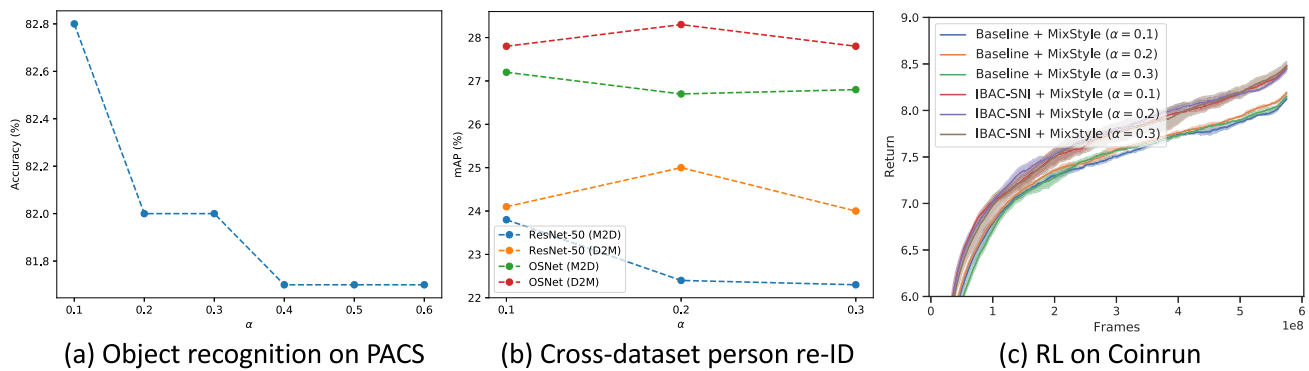


Fig. 6 Evaluating α , the hyper-parameter of the beta distribution (best viewed by zoom-in). M and D in (b) denote Market1501 and Duke, respectively

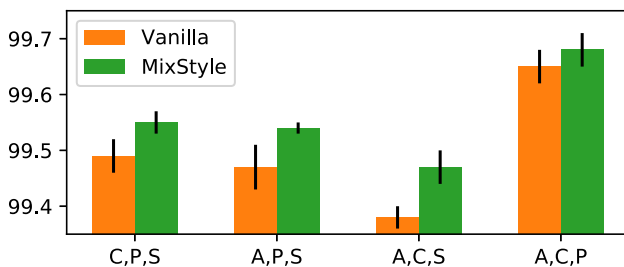


Fig. 7 Test results on the source domains on PACS

Source Domain Performance Fig. 7 justifies that MixStyle does not sacrifice the performance on seen domains in exchange for gains on unseen domains. In fact, the recognition accuracy on the source domains is improved but very slightly (the results are saturated). Such non-trivial gains on source domains well explain the reason why MixStyle works in UDA (Sect. 4.3)—the unlabeled target domain could be viewed as a special “source” since its images are seen during training.

5 Discussion and Conclusion

MixStyle bypasses image synthesis in the pixel space by exploiting the close relation between visual domains and feature statistics to achieve highly-efficient data augmentation. The non-trivial design based on simply mixing feature statistics between instances yields significant improvements to OOD generalization performance—even when only a few labels are available. With a simple modification to the mixing strategy, MixStyle shows potential in dealing with unlabeled data, such as the scenarios of semi-supervised domain generalization and unsupervised domain adaptation.

To unleash the power of MixStyle, we provide practical guidelines on how to integrate MixStyle into a CNN system, e.g., one should apply MixStyle to multiple shallow CNN layers. However, for a new task, it remains relatively unclear exactly to which layers MixStyle should be applied. Nonethe-

less, this is arguably the only “hyper-parameter” that needs to be “tuned”, which is far less than most existing domain generalization methods. To completely eliminate the manual selection process, one could explore, for example, applying MixStyle to every plausible layer and using a differentiable architecture search algorithm to automatically identify the optimal set of layers to choose (Liu et al., 2019; Zhou et al., 2021).

From the application point of view, MixStyle shows encouraging results on a wide variety of problems including object recognition, person re-identification, and reinforcement learning. The extensive experiments conducted in this work suggest that MixStyle can cope with domain shifts related to colors, textures, illuminations, backgrounds, and other visual factors that are associated with image style. This makes sense because based on what we have learned from the image style transfer literature (Huang & Belongie, 2017; Dumoulin et al., 2017), mixing feature statistics can well simulate these visual changes and thus help to learn representations invariant to them. It is worth mentioning that MixStyle has already been used by the community in a variety of AI applications e.g., vehicle re-identification (Huynh, 2021), semantic segmentation (Zhao et al., 2021), 3D point clouds processing (Zhao et al., 2022), medical image analysis [31], speech recognition (Schmid et al., 2022), and so on.

In terms of shortcomings, MixStyle might be less effective in dealing with geometrical shifts, such as rotation or changing viewpoints. This is probably why MixStyle’s gains on Office-Home are less appealing—Fig. 3b shows that the domain shifts on Office-Home are more concerned with viewpoint rather than image style (the bed scenes in the domain of art, product and real-world look similar except viewpoints).

Data Availability The datasets generated during and/or analysed during the current study are available in the project’s github repository, <https://github.com/KaiyangZhou/mixstyle-release>.

References

- Balaji, Y., Sankaranarayanan, S., & Chellappa, R. (2018). Metareg: Towards domain generalization using meta-regularization. In: *NeurIPS*.
- Ben-David, S., Blitzer, J., Crammer, K., Kulesza, A., Pereira, F., & Vaughan, J. W. (2010). A theory of learning from different domains. *ML*.
- Blanchard, G., Lee, G., & Scott, C. (2011). Generalizing from several related classification tasks to a new unlabeled sample. In: *NeurIPS*.
- Carlucci, F. M., D'Innocente, A., Bucci, S., Caputo, B., & Tommasi, T. (2019). Domain generalization by solving jigsaw puzzles. In: *CVPR*.
- Cobbe, K., Klimov, O., Hesse, C., Kim, T., & Schulman, J. (2019). Quantifying generalization in reinforcement learning. In: *ICML*.
- Cubuk, E. D., Zoph, B., Shlens, J., & Le, Q. V. (2019). Randaugment: Practical data augmentation with no separate search. *arXiv preprint arXiv:1909.13719*.
- Deng, Z., Luo, Y., & Zhu, J. (2019). Cluster alignment with a teacher for unsupervised domain adaptation. In: *ICCV*.
- DeVries, T., & Taylor, G.W. (2017). Improved regularization of convolutional neural networks with cutout. *arXiv preprint arXiv:1708.04552*.
- DeVries, T., & Taylor, G.W. (2017). Improved regularization of convolutional neural networks with cutout. *arXiv preprint arXiv:1708.04552*.
- Ding, Z., & Fu, Y. (2017). Deep domain generalization with structured low-rank constraint. *TIP*.
- Dou, Q., Castro, D. C., Kamnitsas, K., & Glocker, B. (2019). Domain generalization via model-agnostic learning of semantic features. In: *NeurIPS*.
- Dumoulin, V., Shlens, J., & Kudlur, M. (2017). A learned representation for artistic style. In: *ICLR*.
- Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., & Dunning, I., et al. (2018). Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In: *ICML*.
- Fan, Q., Segu, M., Tai, Y. W., Yu, F., Tang, C. K., Schiele, B., & Dai, D. (2022). Normalization perturbation: A simple domain generalization method for real-world domain shifts. *arXiv preprint arXiv:2211.04393*.
- Farebrother, J., Machado, M. C., & Bowling, M. (2018). Generalization and regularization in dqn. *arXiv preprint arXiv:1810.00123*.
- Gamrian, S., & Goldberg, Y. (2019). Transfer learning for related reinforcement learning tasks via image-to-image translation. In: *ICML*.
- Ganin, Y., & Lempitsky, V. S. (2015). Unsupervised domain adaptation by backpropagation. In: *ICML*.
- Ghiasi, G., Lin, T. Y., & Le, Q. V. (2018). Dropblock: A regularization method for convolutional networks. In: *NeurIPS*.
- Gong, R., Li, W., Chen, Y., & Van Gool, L. (2019). Dlow: Domain flow for adaptation and generalization. In: *CVPR*.
- Gulrajani, I., & Lopez-Paz, D. (2020). In search of lost domain generalization. *arXiv preprint arXiv:2007.01434*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In: *CVPR*.
- Hoffman, J., Tzeng, E., Park, T., Zhu, J. Y., Isola, P., Saenko, K., Efros, A., & Darrell, T. (2018). Cycada: Cycle-consistent adversarial domain adaptation. In: *ICML*.
- Huang, X., & Belongie, S. (2017). Arbitrary style transfer in real-time with adaptive instance normalization. In: *ICCV*.
- Huynh, S. V. (2021). A strong baseline for vehicle re-identification. In: *CVPR-W*.
- Igl, M., Ciosek, K., Li, Y., Tschitschek, S., Zhang, C., Devlin, S., & Hofmann, K. (2019). Generalization in reinforcement learning with selective noise injection and information bottleneck. In: *NeurIPS*.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: *ICML*.
- Justesen, N., Torrado, R. R., Bontrager, P., Khalifa, A., Togelius, J., & Risi, S. (2018). Illuminating generalization in deep reinforcement learning through procedural level generation. *arXiv preprint arXiv:1806.10729*.
- Kang, G., Jiang, L., Wei, Y., Yang, Y., & Hauptmann, A. G. (2020). Contrastive adaptation network for single-and multi-source domain adaptation. *TPAMI*.
- Kostrikov, I., Yarats, D., & Fergus, R. (2021). Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In: *ICLR*.
- Krizhevsky, A., Sutskever, I., & Hinton, G.E. (2012). Imagenet classification with deep convolutional neural networks. In: *NeurIPS*.
- Kushibar, K., & Jouide, S. Cancer radiomics extraction and selection pipeline.
- Laskin, M., Lee, K., Stooke, A., Pinto, L., Abbeel, P., & Srinivas, A. (2020). Reinforcement learning with augmented data. In: *NeurIPS*.
- Lee, C.Y., Batra, T., Baig, M. H., & Ulbricht, D. (2019). Sliced wasserstein discrepancy for unsupervised domain adaptation. In: *CVPR*.
- Lee, K., Lee, K., Shin, J., & Lee, H. (2020). Network randomization: A simple technique for generalization in deep reinforcement learning. In: *ICLR*.
- Li, D., Yang, Y., Song, Y.Z., & Hospedales, T. M. (2017). Deeper, broader and artier domain generalization. In: *ICCV*.
- Li, D., Yang, Y., Song, Y.Z., & Hospedales, T. M. (2018). Learning to generalize: Meta-learning for domain generalization. In: *AAAI*.
- Li, D., Zhang, J., Yang, Y., Liu, C., Song, Y.Z., & Hospedales, T. M. (2019). Episodic training for domain generalization. In: *ICCV*.
- Li, H., Jialin Pan, S., Wang, S., & Kot, A. C. (2018). Domain generalization with adversarial feature learning. In: *CVPR*.
- Li, Y., Tiana, X., Gong, M., Liu, Y., Liu, T., Zhang, K., & Tao, D. (2018). Deep domain generalization via conditional invariant adversarial networks. In: *ECCV*.
- Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C.L. (2014). Microsoft coco: Common objects in context. In: *ECCV*.
- Liu, H., Simonyan, K., & Yang, Y. (2019). Darts: Differentiable architecture search. In: *ICLR*.
- Liu, Z., Miao, Z., Pan, X., Zhan, X., Lin, D., Yu, S. X., & Gong, B. (2020). Open compound domain adaptation. In: *CVPR*.
- Long, M., Cao, Y., Wang, J., & Jordan, M. I. (2015). Learning transferable features with deep adaptation networks. In: *ICML*.
- Long, M., Zhu, H., Wang, J., & Jordan, M. I. (2016). Unsupervised domain adaptation with residual transfer networks. In: *NeurIPS*.
- Loshchilov, I., & Hutter, F. (2017). Sgdr: Stochastic gradient descent with warm restarts. In: *ICLR*.
- Lu, Z., Yang, Y., Zhu, X., Liu, C., Song, Y. Z., & Xiang, T. (2020). Stochastic classifiers for unsupervised domain adaptation. In: *CVPR*.
- Maaten, L.v.d., & Hinton, G. (2008). Visualizing data using t-sne. *JMLR*.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*.
- Motian, S., Piccirilli, M., Adjero, D. A., & Doretto, G. (2017). Unified deep supervised domain adaptation and generalization. In: *ICCV*.
- Muandet, K., Balduzzi, D., & Scholkopf, B. (2013). Domain generalization via invariant feature representation. In: *ICML*.
- Peng, X., Bai, Q., Xia, X., Huang, Z., Saenko K., & Wang, B. (2019). Moment matching for multi-source domain adaptation. In: *ICCV*.
- Peng, X., Usman, B., Kaushik, N., Hoffman, J., Wang, D., & Saenko, K. (2017). Visda: The visual domain adaptation challenge. *arXiv preprint arXiv:1710.06924*.

- Ristani, E., Solera, F., Zou, R., Cucchiara, R., & Tomasi, C. (2016). Performance measures and a data set for multi-target, multi-camera tracking. In: ECCV.
- Saito, K., Watanabe, K., Ushiku, Y., & Harada, T. (2018). Maximum classifier discrepancy for unsupervised domain adaptation. In: CVPR.
- Schmid, F., Masoudian, S., Koutini, K., & Widmer, G. (2022). Cp-jku submission to dcase22: Distilling knowledge for low-complexity convolutional neural networks from a patchout audio transformer. Tech. rep., DCASE2022 Challenge, Tech. Rep.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint [arXiv:1707.06347](https://arxiv.org/abs/1707.06347).
- Segu, M., Tonioni, A., & Tombari, F. (2023). Batch normalization embeddings for deep domain generalization. *Pattern Recognition*, 135, 109115.
- Seo, S., Suh, Y., Kim, D., Kim, G., Han, J., & Han, B. (2020). Learning to optimize domain specific normalization for domain generalization. In: ECCV.
- Shankar, S., Piratla, V., Chakrabarti, S., Chaudhuri, S., Jyothi, P., & Sarawagi, S. (2018). Generalizing across domains via cross-gradient training. In: ICLR.
- Sohn, K., Berthelot, D., Li, C.L., Zhang, Z., Carlini, N., Cubuk, E. D., Kurakin, A., Zhang, H. & Raffel, C. (2020). Fixmatch: Simplifying semi-supervised learning with consistency and confidence. In: NeurIPS.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *JMLR*.
- Sun, B., & Saenko, K. (2016). Deep coral: Correlation alignment for deep domain adaptation. In: ECCV.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015). Going deeper with convolutions. In: CVPR.
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In: IROS.
- Tzeng, E., Hoffman, J., Saenko, K., & Darrell, T. (2017). Adversarial discriminative domain adaptation. In: CVPR.
- Ulyanov, D., Vedaldi, A., & Lempitsky, V. (2016). Instance normalization: The missing ingredient for fast stylization. [arXiv:1607.08022](https://arxiv.org/abs/1607.08022).
- Venkateswara, H., Eusebio, J., Chakraborty, S., & Panchanathan, S. (2017). Deep hashing network for unsupervised domain adaptation. In: CVPR.
- Verma, V., Lamb, A., Beckham, C., Najafi, A., Mitliagkas, I., Lopez-Paz, D., & Bengio, Y. (2019). Manifold mixup: Better representations by interpolating hidden states. In: ICML.
- Wang, H., Xu, M., Ni, B., & Zhang, W. (2020). Learning to combine: Knowledge aggregation for multi-source domain adaptation. In: ECCV.
- Wang, S., Yu, L., Li, C., Fu, C.W., & Heng, P.A. (2020). Learning from extrinsic and intrinsic supervisions for domain generalization. In: ECCV.
- Xu, R., Chen, Z., Zuo, W., Yan, J., & Lin, L. (2018). Deep cocktail network: Multi-source unsupervised domain adaptation with category shift. In: CVPR.
- Yun, S., Han, D., Oh, S.J., Chun, S., Choe, J., & Yoo, Y. (2019). Cutmix: Regularization strategy to train strong classifiers with localizable features. In: ICCV.
- Zhang, C., Vinyals, O., Munos, R., & Bengio, S. (2018). A study on overfitting in deep reinforcement learning. arXiv preprint [arXiv:1804.06893](https://arxiv.org/abs/1804.06893).
- Zhang, H., Cisse, M., Dauphin, Y.N., & Lopez-Paz, D. (2018). Mixup: Beyond empirical risk minimization. In: ICLR.
- Zhao, H., Zhang, S., Wu, G., Moura, J. M., Costeira, J. P., & Gordon, G. J. (2018). Adversarial multiple source domain adaptation. In: NeurIPS.
- Zhao, S., Wang, G., Zhang, S., Gu, Y., Li, Y., Song, Z., Xu, P., Hu, R., Chai, H., & Keutzer, K. (2020). Multi-source distilling domain adaptation. In: AAAI.
- Zhao, Y., Zhong, Z., Luo, Z., Lee, G.H., & Sebe, N. (2021). Source-free open compound domain adaptation in semantic segmentation. arXiv preprint [arXiv:2106.03422](https://arxiv.org/abs/2106.03422).
- Zhao, Z., Wu, Z., Wu, X., Zhang, C., & Wang, S. (2022). Crossmodal few-shot 3d point cloud semantic segmentation. In: Proceedings of the 30th ACM international conference on multimedia (pp. 4760–4768).
- Zheng, L., Shen, L., Tian, L., Wang, S., Wang, J., & Tian, Q. (2015). Scalable person re-identification: A benchmark. In: ICCV.
- Zheng, Z., Zheng, L., & Yang, Y. (2017). Unlabeled samples generated by gan improve the person re-identification baseline in vitro. In: ICCV.
- Zhong, Z., Zheng, L., Kang, G., Li, S., & Yang, Y. (2020). Random erasing data augmentation. In: AAAI.
- Zhou, K., Liu, Z., Qiao, Y., Xiang, T., & Loy, C.C. (2021). Domain generalization: A survey. arXiv preprint [arXiv:2103.02503](https://arxiv.org/abs/2103.02503).
- Zhou, K., Loy, C. C., & Liu, Z. (2021). Semi-supervised domain generalization with stochastic stylematch. arXiv preprint [arXiv:2106.00592](https://arxiv.org/abs/2106.00592).
- Zhou, K., & Xiang, T. (2019). Torchreid: A library for deep learning person re-identification in pytorch. arXiv preprint [arXiv:1910.10093](https://arxiv.org/abs/1910.10093).
- Zhou, K., Yang, Y., Cavallaro, A., & Xiang, T. (2021). Learning generalisable omni-scale representations for person re-identification. TPAMI.
- Zhou, K., Yang, Y., Hospedales, T. & Xiang, T. (2020). Learning to generate novel domains for domain generalization. In: ECCV.
- Zhou, K., Yang, Y., Hospedales, T.M. & Xiang, T. (2020a). Deep domain-adversarial image generation for domain generalisation. In: AAAI.
- Zhou, K., Yang, Y., Qiao, Y., & Xiang, T. (2020). Domain adaptive ensemble learning. arXiv preprint [arXiv:2003.07325](https://arxiv.org/abs/2003.07325).
- Zhou, K., Yang, Y., Qiao, Y., & Xiang, T. (2021). Domain generalization with mixstyle. In: ICLR.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript or version of this article is solely governed by the terms of such publishing agreement and applicable law.